Matrix Algebra and MATLAB

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk

Outline

1. Matrices in MATLAB
   - Creating Vector and Matrix Variables
2. Matrices
   - Some useful operations on matrices
3. Matrix Algebra
   - Matrices Multiplication
   - Determinant of a matrix
   - Inverse of a matrix
4. Exercise

Outline

1. Matrices in MATLAB
   - Creating Vector and Matrix Variables
2. Matrices
   - Some useful operations on matrices
3. Matrix Algebra
   - Matrices Multiplication
   - Determinant of a matrix
   - Inverse of a matrix
4. Exercise

Matrices in MATLAB
Matrices
Matrix Algebra
Exercise

Outline

1. Matrices in MATLAB
   - Creating Vector and Matrix Variables
2. Matrices
   - Some useful operations on matrices
3. Matrix Algebra
   - Matrices Multiplication
   - Determinant of a matrix
   - Inverse of a matrix
4. Exercise

Outline

1. Matrices in MATLAB
   - Creating Vector and Matrix Variables
2. Matrices
   - Some useful operations on matrices
3. Matrix Algebra
   - Matrices Multiplication
   - Determinant of a matrix
   - Inverse of a matrix
4. Exercise

Matrices can be introduced into MATLAB in several ways:

- Entered by an explicit list of elements.
- Generated by so called built–in statements and functions.
- Loaded by external data.
- Created by M–Files.

Generating matrices as a list of its elements. The following basis conventions have to be taken into account:

- The elements of a row are separated by commas or spaces.
- A semi-colon indicates the end of each row.
- The entire list of elements has to be surrounded by brackets [ ].

Row Vector

- Row vector variables can be created in several ways.
- The simplest method is to put the values that you want in the variable in square brackets, separated by either spaces or commas.
- For instance, a row vector with four elements in MATLAB can be generated as:
- rowvec=[2,4,5,6]
- The required output is as under
- rowvec =
  2 4 5 6

Row vector using Colon Operator

- Row vector can also be generated using Colon operator(;)
- The entries in the generated Row vector iterates from the starting to the ending value with a default step of one.
- In this case the square brackets are not necessary.
- For example, a row vector having $2, 3, 4, 5, 6$ entries can be generated as
- » colvec=2:6
- The necessary output is
- colvec =
  2 3 4 5 6

Colon Operator with Step value

- Row vector with Colon operator can also be generated by defining step value for each entry of the matrix.
- For example, the row vector with entries $2, 4, 6, 8, 10$ can be generated as
- » stepvec = 2:2:10
- The desired output is
- stepvec =
  2 4 6 8 10

- Row vector is generated in different ways

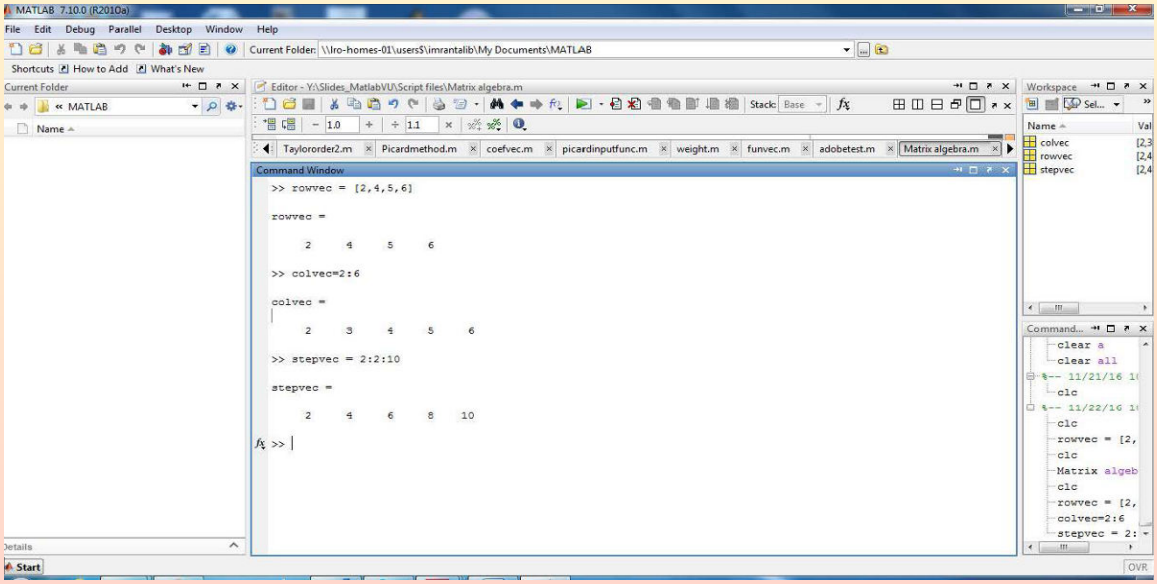


Figure: Display of the Row vector in different ways

Column Vector

- There are two basic methods for creating Column vector:
- Either by putting the values in square brackets, separated by semicolons.
- or by creating a row vector and then transposing it.
- The transpose operator in MATLAB is the apostrophe.

Column Vector with Square bracket and semicolons

- For example, the column vector with entries $2, 5, 10$ can be generated as
- » colvec=[2;5;10]
- The required output is
  colvec =
  2
  5
  10

Column Vector with apostrophe

- Alternatively, the column vector with entries $2, 5, 10$ can also be generated as
- » Rowvec=[2,5,10]
  Rowvec =
  2 5 10
  » colvec=Rowvec'
  colvec =
  2
  5
  10

- Column vector is generated in different ways

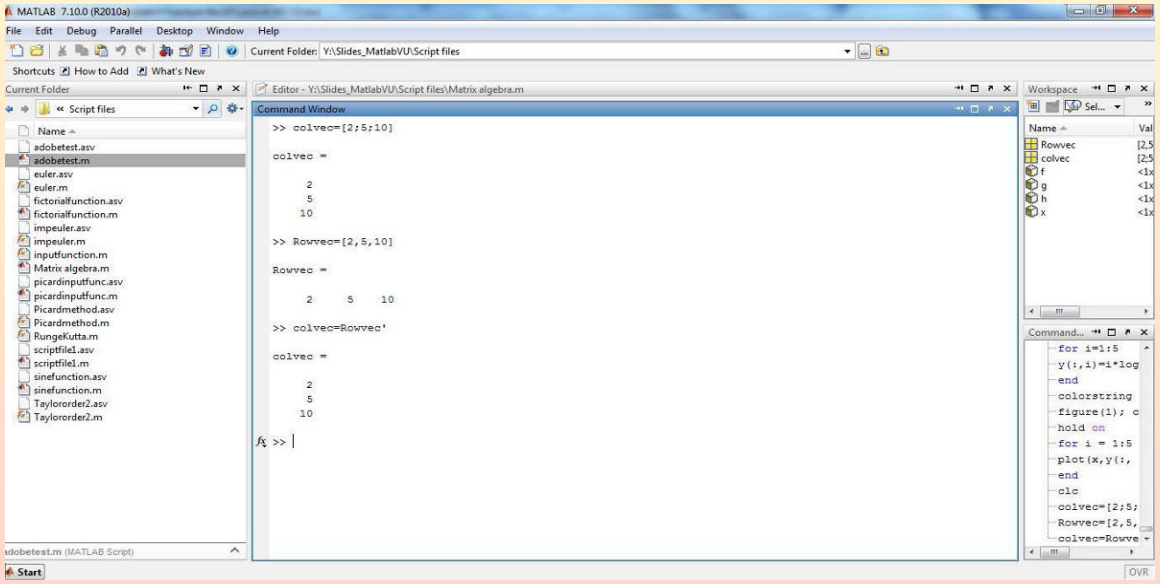


Figure: Display of the Column vector in different ways

- Matrix variables can be created by putting the values in square brackets.
- The values within the rows are separated by either spaces or commas.
- Each individual row is separated by semicolons.
- The must always be the same number of values in every row.
- The colon operator can be used to iterate within the rows.

- For example, the matrix

$$A = \begin{bmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{bmatrix}$$

in MATLAB is generated as

- » A=[5/6,1/6,0;5/6,0,1/6;0,5/6,1/6]
  A =
  0.8333 0.1667 0
  0.8333 0 0.1667
  0 0.8333 0.1667

or

- A=sym([5/6,1/6,0;5/6,0,1/6;0,5/6,1/6])
  A =

$$\begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$$

- Generation of the matrix in MATLAB

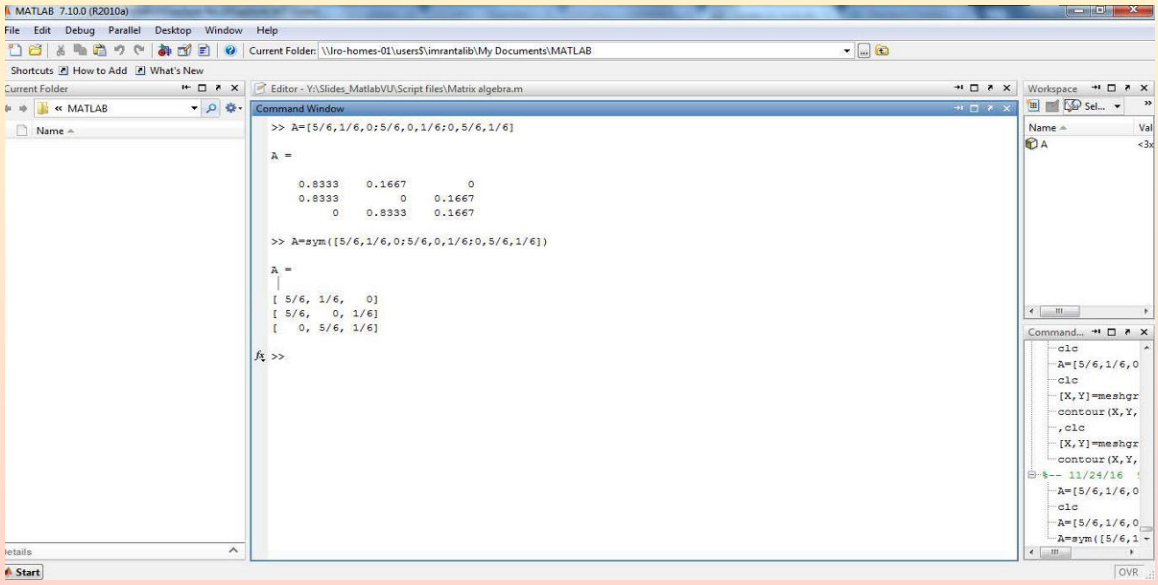


Figure: Graphical view of the matrices

Sum of column vectors of a matrix

- The **sum** command computes a **Row vector** containing the sum of the **columns** of a matrix.
- Note that MATLAB prefers to work with the columns of a matrix.
- For example, the sum of the columns of the matrix A is computed as:
-
- A=sym([5/6,1/6,0;5/6,0,1/6;0,5/6,1/6])

A =

$$\begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$$

» B = sum(A)

B =

$$\begin{pmatrix} \frac{5}{3} & 1 & \frac{1}{3} \end{pmatrix}$$

Sum of row vectors of a matrix

- To compute the sum of Row vectors of a matrix, we should take the transpose, then compute the sum and finally transpose the result.
- For example, the sum of the **Row** vectors of the matrix A is calculated in the next slide. Please see it.

- Sum of the entries of the column vectors of A

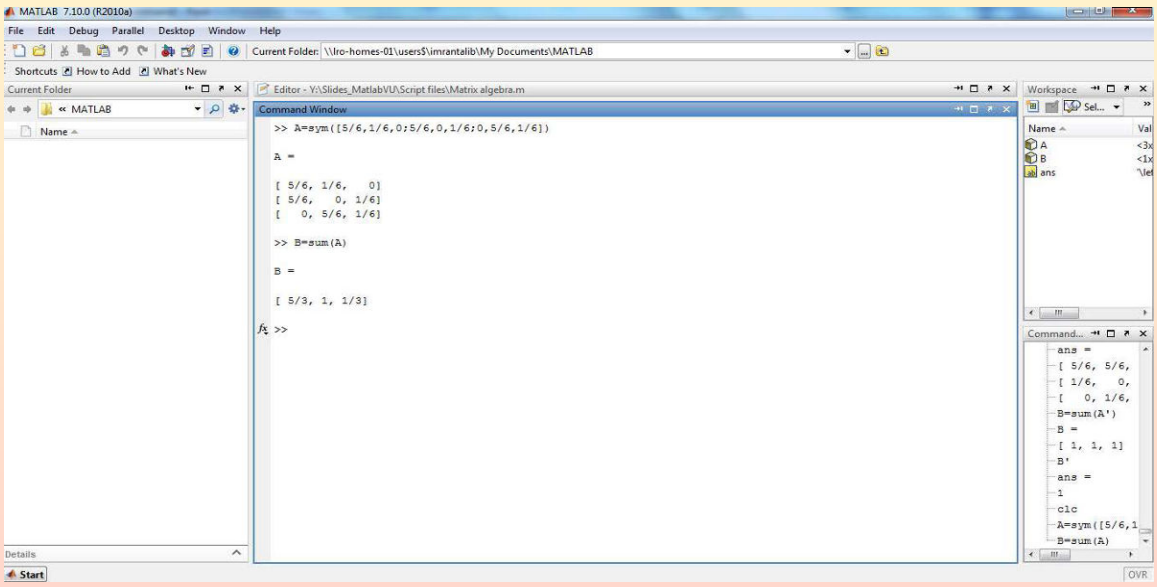


Figure: Display of the sum command

Sum of row vectors of a matrix

- »A = $\begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$

  » A'= $\begin{pmatrix} \frac{5}{6} & \frac{5}{6} & 0 \\ \frac{1}{6} & 0 & \frac{5}{6} \\ 0 & \frac{1}{6} & \frac{1}{6} \end{pmatrix}$

  » B=sum(A')

  B =

  [1, 1, 1]

  » B'

  ans =

  1

  1

  1.

- Sum of the entries of the row vectors of A

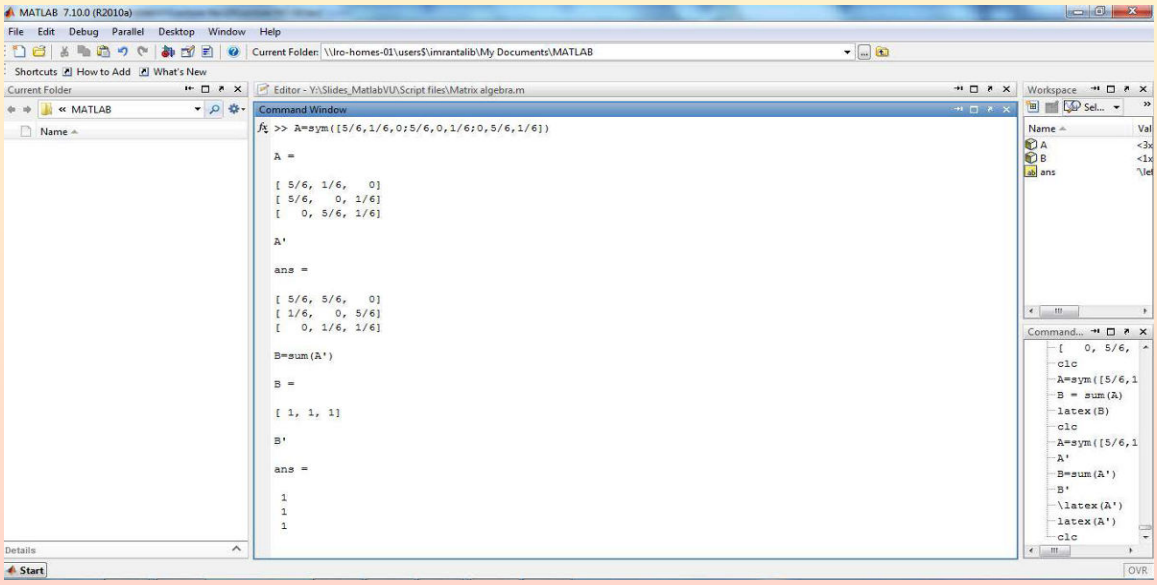


Figure: Display of the sum command

Column vector containing the Diagonal elements of a matrix

- The **diag** command produces a column vector containing the elements of the main diagonal of a matrix.
- For example,
- »A = $\begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$
- The MATLAB displays the following result:
- » B=diag(A)
  B =
  5/6
  0
  1/6

Sum of individual row and column

- The collective use of **sum** command and **colon (:)** operator is used to compute the sum of any individual column and row of a matrix.
- The colon operator refers to all elements in a row or column of a matrix.
- The key word **end** refers to last column or row.
- For example, the sum of last column of the matrix *A* in MATLAB is calculated as
- »A = $\begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$
- The MATLAB displays the following result:
- » B=sum(A(:,end))
  B =
  1/3

- Sum of the entries of the last and the second column of A

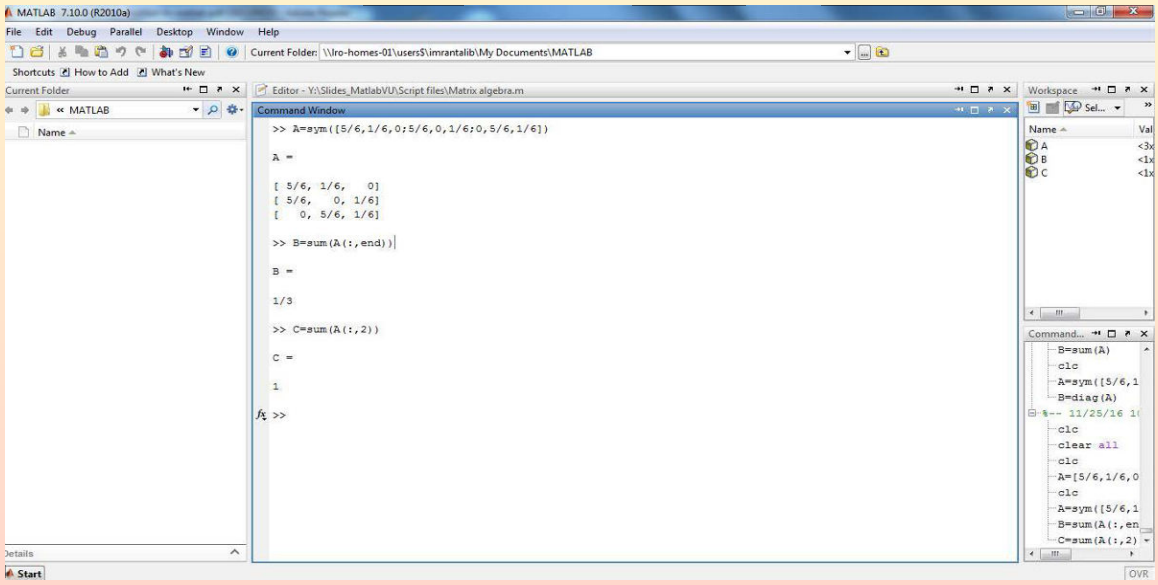


Figure: Display of the sum command with colon operator

Matrices multiplication using for loop

- The matrix multiplication can be defined using MATLAB's **for** loop, colon notation, and vector scalar product.
- For example, the MATLAB syntax for the multiplication of two matrices A and B using for loop is as under:
- clear all;
  close all;
  clc;
  A=[1,2,3;4,5,6];
  B=[1,4;6,7;0,1];
  for i=1:2
  for j=1:2
  C(i,j)=A(i,:)*B(:,j);
  end
  end
  C

- Multiplication of the matrices A and B

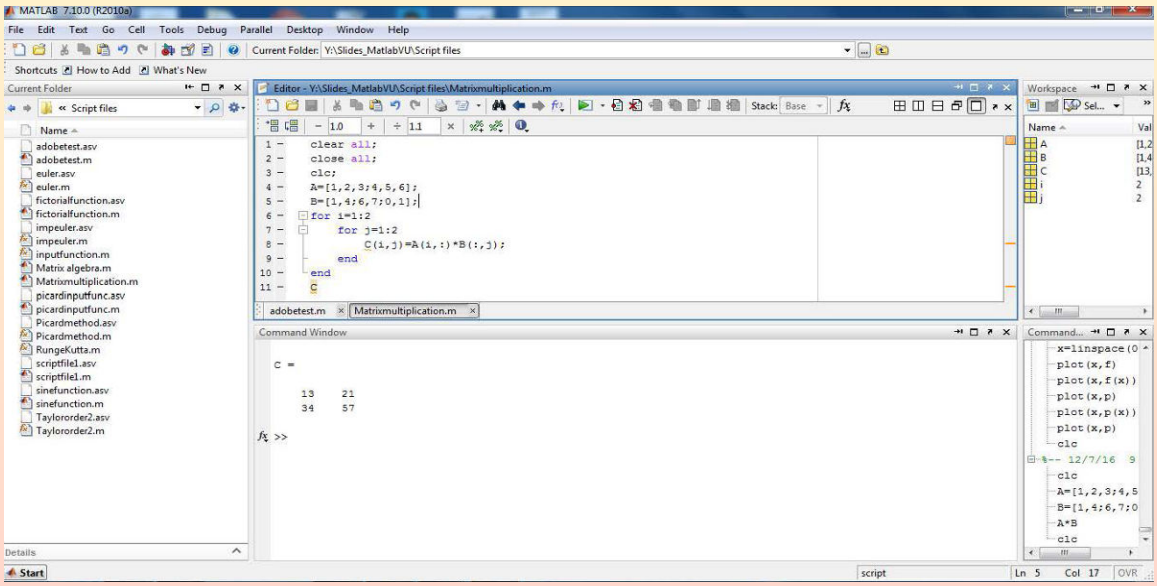


Figure: Matrices multiplication using for loop

Matrices multiplication using asterisk operator

- Single asterisk operator (*) can also be used for the matrices multiplication.
- For example, above defined matrices, A and B can also be multiplied in MATLAB as
- » A=[1,2,3;4,5,6];
  » B=[1,4;6,7;0,1];
  »
  » C=A*B.

- Multiplication of the matrices A and B.

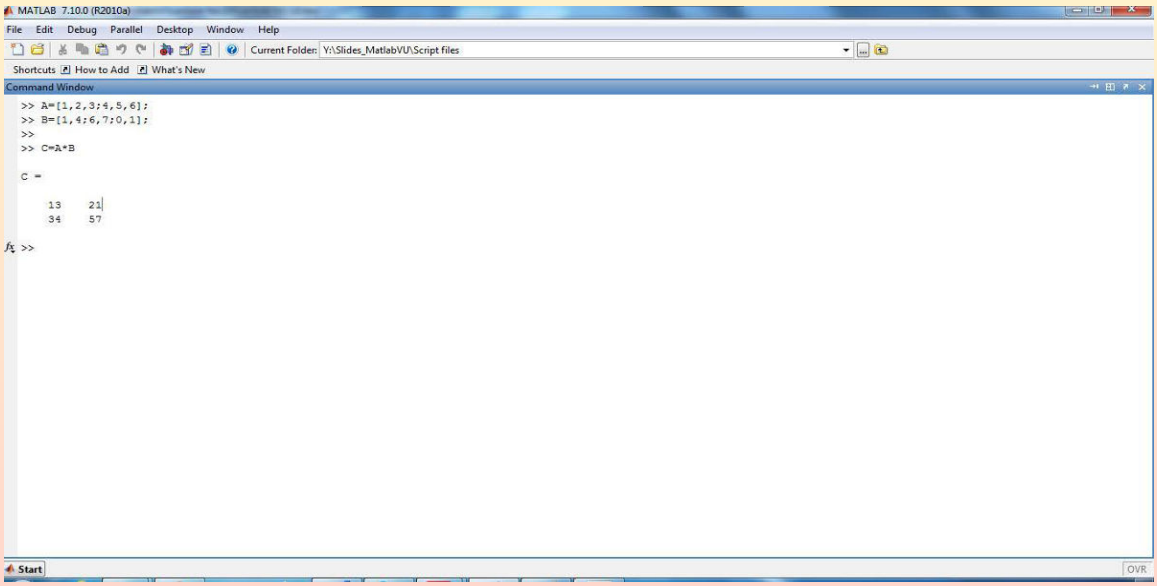


Figure: Matrices multiplication using asterisk operator

- The determinant of a square matrix in MATLAB can be determined using **det** command.
- For example, the determinant of the following matrix is determined as
- »A =
$$\begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{5}{6} & 0 & \frac{1}{6} \\ 0 & \frac{5}{6} & \frac{1}{6} \end{pmatrix}$$
- » A=sym([5/6,1/6,0;1/6,0,5/6;0,5/6,1])
- » det(A)
- The desired MATLAB output is
ans =
-131/216

- The inverse of a square matrix in MATLAB is determined using the **inv** command.
- For example, the inverse of the matrix A is computed as
- » inv(A)
- The required output is as under:
- ans =

$$\begin{pmatrix} \frac{150}{131} & \frac{36}{131} & -\frac{30}{131} \\ \frac{36}{131} & -\frac{180}{131} & \frac{150}{131} \\ -\frac{30}{131} & \frac{150}{131} & \frac{6}{131} \end{pmatrix}$$

- Determinant and inverse of the matrix A.

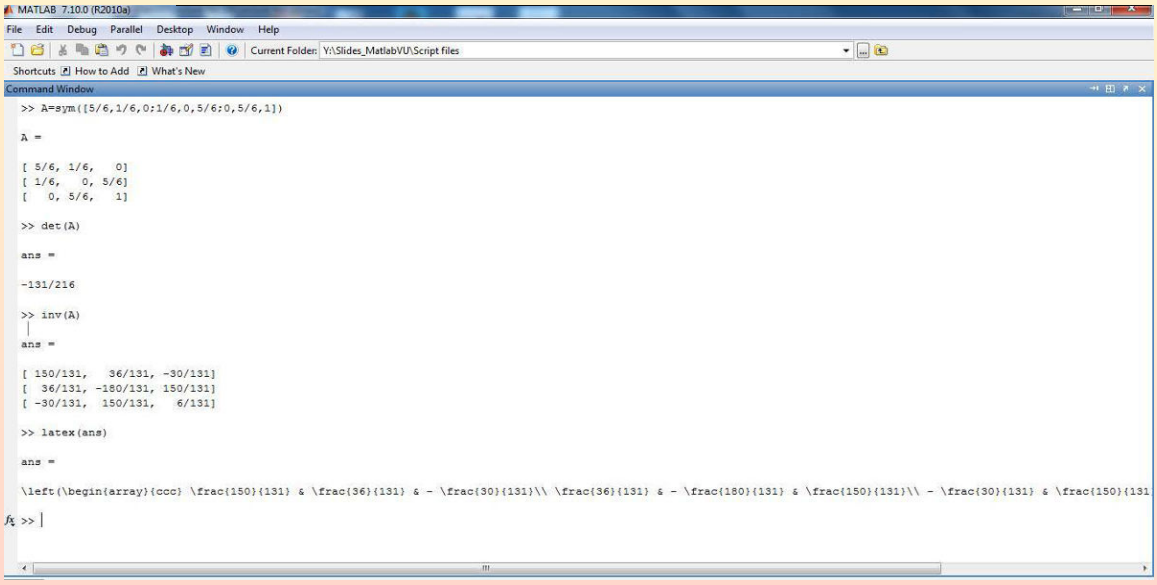


Figure: det and inv command in MATLAB

Compute AB using for loop in MATLAB, if

- A= $\begin{pmatrix} \frac{5}{6} & \frac{1}{6} & 0 \\ \frac{1}{6} & 0 & \frac{5}{6} \\ 0 & \frac{5}{6} & 1 \end{pmatrix}$
- B= $\begin{pmatrix} \frac{5}{2} & \frac{1}{2} & 0 \\ \frac{1}{3} & 0 & \frac{5}{3} \\ 0 & \frac{5}{6} & 1 \end{pmatrix}$

Compute the sum of the second column of the following matrix using **sum** command.

- C= $\begin{pmatrix} 1 & \frac{1}{2} & 0 \\ \frac{1}{3} & \frac{1}{6} & 2 \\ 0 & 0 & 1 \end{pmatrix}$
- Compute the sum of the second row of the matrix C using **sum** command.

Linear Algebra and MATLAB

Department of Mathematics and Statistics
Virtual University of Pakistan, Lahore, Pakistan.

imrantalib@vu.edu.pk

Outline

1. Eigen Values and Eigen Vectors
2. Reduced row echelon form
3. Solution of system of linear equations
   - Solution of equations when coefficient matrix have inverse
4. Elementary row operations
5. Spanning set
6. Exercise

Outline

1. Eigen Values and Eigen Vectors
2. Reduced row echelon form
3. Solution of system of linear equations
   - Solution of equations when coefficient matrix have inverse
4. Elementary row operations
5. Spanning set
6. Exercise

Eigen Values and Eigen Vectors
Reduced row echelon form
Solution of system of linear equations
Elementary row operations
Spanning set
Exercise

Outline

1. Eigen Values and Eigen Vectors
2. Reduced row echelon form
3. Solution of system of linear equations
   - Solution of equations when coefficient matrix have inverse
4. Elementary row operations
5. Spanning set
6. Exercise

Outline

1. Eigen Values and Eigen Vectors
2. Reduced row echelon form
3. Solution of system of linear equations
   - Solution of equations when coefficient matrix have inverse
4. Elementary row operations
5. Spanning set
6. Exercise

Eigen Values and Eigen Vectors
Reduced row echelon form
Solution of system of linear equations
Elementary row operations
Spanning set
Exercise

Outline

1. Eigen Values and Eigen Vectors
2. Reduced row echelon form
3. Solution of system of linear equations
   - Solution of equations when coefficient matrix have inverse
4. Elementary row operations
5. Spanning set
6. Exercise

Outline

1. Eigen Values and Eigen Vectors
2. Reduced row echelon form
3. Solution of system of linear equations
   - Solution of equations when coefficient matrix have inverse
4. Elementary row operations
5. Spanning set
6. Exercise

- The **eig** command is used to determine the Eigen value of a square matrix.
- This command with slightly modification also used to determine the associated Eigen vector.
- Consider the following example:

Example:

- Compute the eigen values and eigen vectors for the following matrix:

$$B = \begin{pmatrix} 0 & 1 & 3 \\ 2 & 4 & 1 \\ 6 & 1 & 8 \end{pmatrix}$$

- The recipe of the solution is as under:
- B=[0,1,3;2,4,1;6,1,8];
  eig(B)
  $[P, D] = eig(B)$

Note:

- The columns of the matrix P denotes the eigen vectors of the matrix B.
- The matrix D denotes the diagonal matrix having eigen values on the main diaginal.

- Computation of the eigen values and eigen vectors.

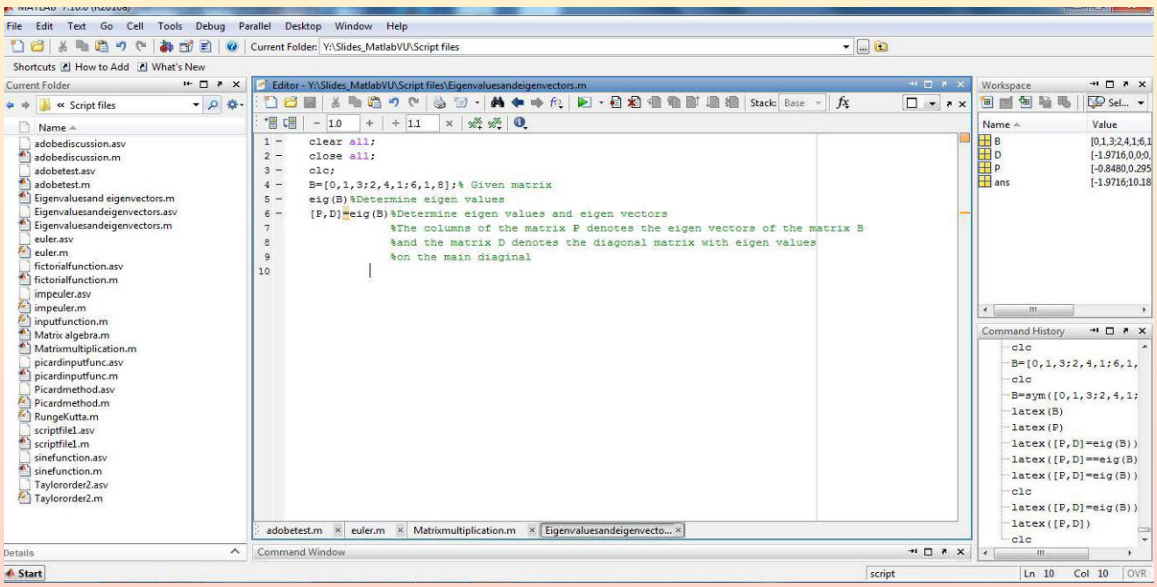


Figure: Editor file having code of eigen values and eigen vectors

- Results of the eigen values and eigen vectors.

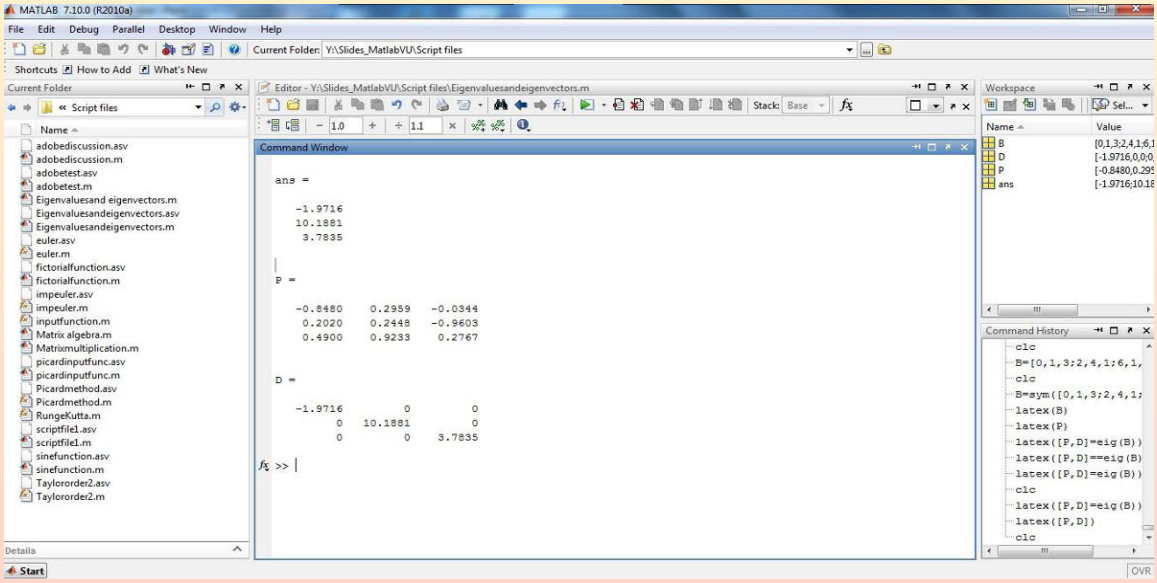


Figure: Command window having outputs of the eigen values and eigen vectors

- MATLAB also gives the reduced echelon form of a matrix.
- For this **rref** command is used.
- Consider the following example:

Example:

- Find the reduced echelon form of the following matrix:

$$B = \begin{pmatrix} 0 & 1 & 3 \\ 2 & 4 & 1 \\ 6 & 1 & 8 \end{pmatrix}$$

- Reduced echelon form.

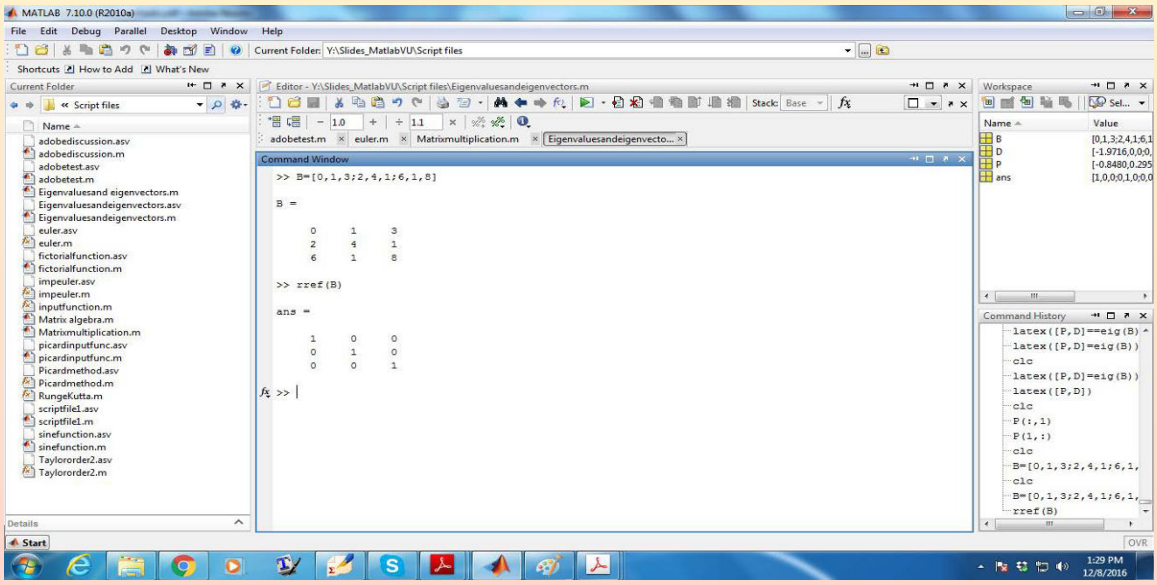


Figure: output of **rref** command

Solution of equations when coefficient matrix have inverse

- While dealing with system of linear equations two situation often occurs:
- Whether the coefficient matrix have inverse or have not?
- First we discuss the case when the coefficient matrix have inverse

Example:

- Find the solution of the following system of linear equations:

$$\begin{aligned} x_1 + 3x_2 &= -2 \\ 2x_1 + 4x_2 &= 1. \end{aligned} \tag{3.1}$$

Solution:

- The coefficient matrix is as under:
  $A = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix}$
- The column matrix is given below:
  $B = \begin{pmatrix} -2 \\ 1 \end{pmatrix}$

- The MATLAB code for the solution of the system is as under:
- A=sym([1,3;2,4]);
  B=sym([-2;1]);
  C=inv(A);
  X=C*B
- For more clarity see the next slide.

- Solution of linear equations in MATLAB when coefficient matrix have inverse.

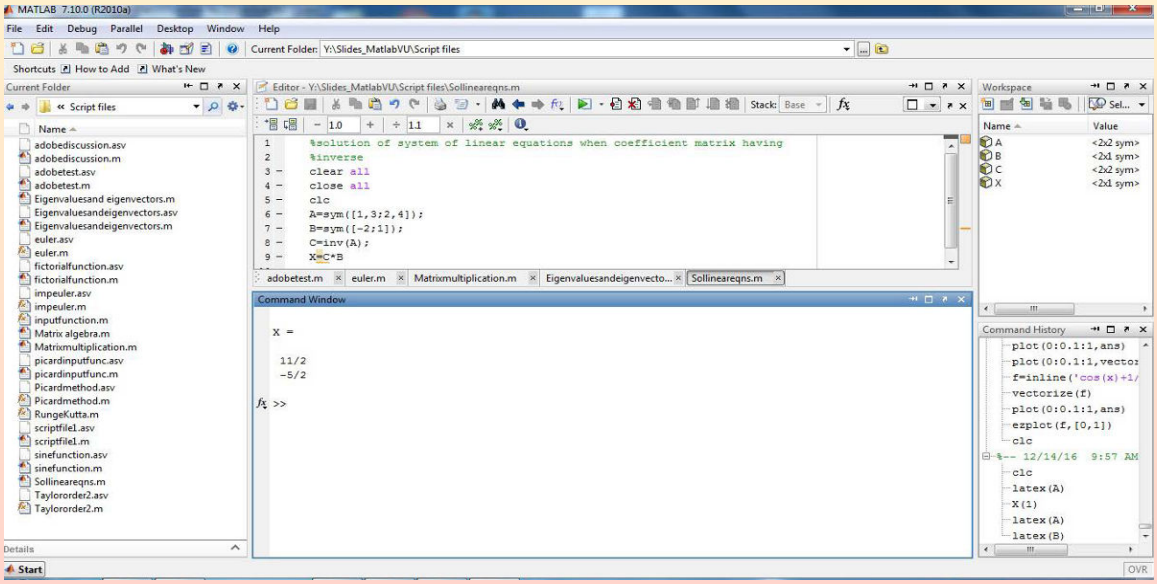


Figure: Display of the MATLAB Editor with code of the solution of linear equations

Example:

- Solve the following system of linear equations using **solve** command:

$$\begin{aligned} 3x + 2y + z &= 1, \\ y &= 2, \\ x + 2y &= 3. \end{aligned} \tag{3.2}$$

Solution:

- The MATLAB code for the solution is:
- eqn1='3*x+2*y+z=1';
  eqn2='y=2';
  eqn3='x+2*y=3';
  sol=solve(eqn1,eqn2,eqn3);
  X=double([sol.x,sol.y,sol.z])'

- Solution of linear equations

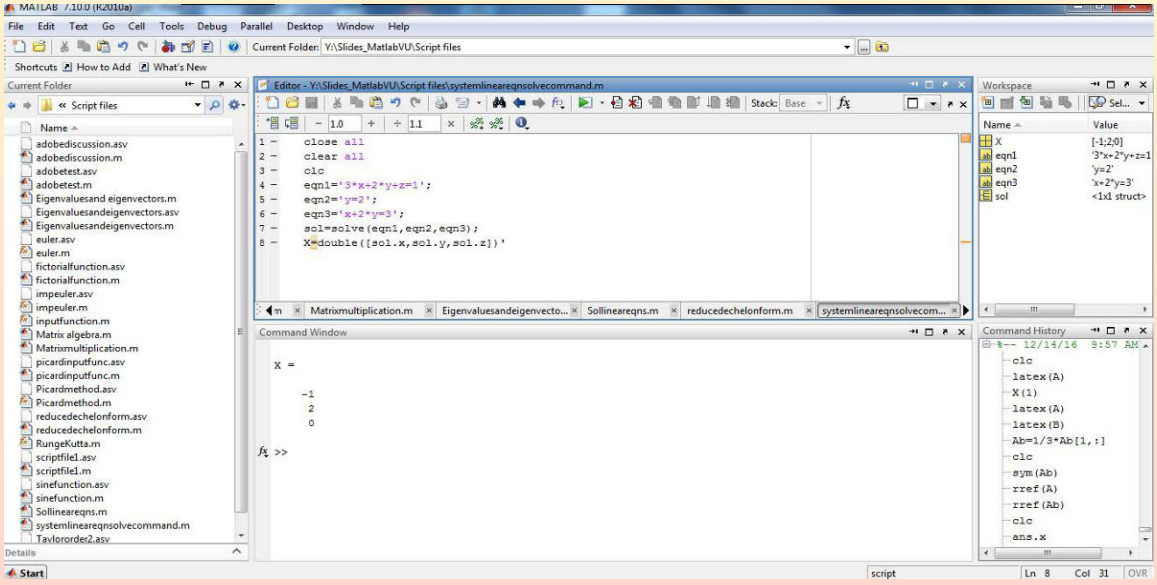


Figure: Display of the MATLAB Editor with code of the solution of linear equations using solve command

- The matrix can also be converted into reduced echelon form using row operations.
- Consider the following example:

Example:

- Consider the system of linear equations:

$$\begin{aligned} 3x + 2y + z &= 1, \\ y &= 2, \\ x + 2y &= 3. \end{aligned} \tag{4.1}$$

- Enter it into MATLAB as an augmented matrix named *Ab*.
- Use elementary row operations to reduce it to reduced-echelon form.

Solution:

- The MATLAB recipe for the solution is as under:
- A=[3,2,1;0,1,0;1,2,0];
  b=[1,2,3]';
  Ab=[A b];
  Ab(1,:)=1/3*Ab(1,:);
  Ab(3,:)=Ab(3,:)-Ab(1,:);
  Ab(1,:)=Ab(1,:)-0.6667*Ab(2,:);
  Ab(3,:)=Ab(3,:)-1.3333*Ab(2,:);
  Ab(3,:)=1/-0.3333*Ab(3,:);
  Ab(1,:)=Ab(1,:)-0.3333*Ab(3,:)

- Reduced echelon form using row operations

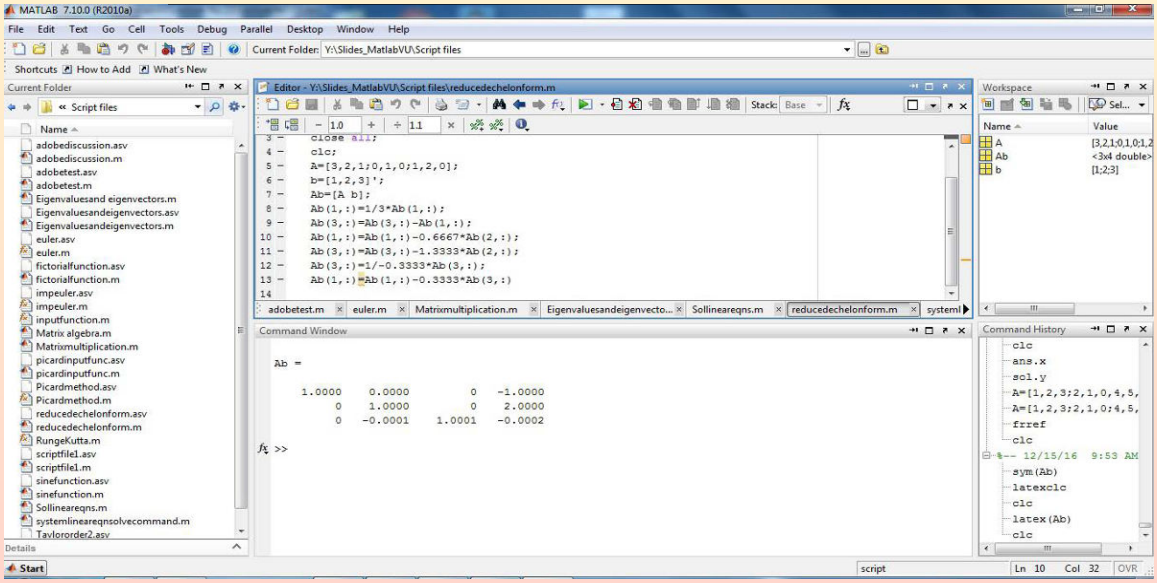


Figure: Reduction of the matrix into reduced echelon form

- **rref** command is also used to check that whether a vector can be written as a linear combination of the elements (vectors) of a set or not?
- Consider the following example:

Example:

- Show that $\begin{pmatrix} a \\ b \end{pmatrix}$ is in the span of $\begin{pmatrix} -2 & 1 \\ 1 & 6 \end{pmatrix}$ for any a and b.

Solution:

- First declare that a and b as symbolic.
- Enter the matrix Ab that can help you to show this.
- Use **rref** on the matrix Ab.
- Explicitly give the weights w_1 and w_2, such that:
  $\begin{pmatrix} a \\ b \end{pmatrix} = w_1 \begin{pmatrix} -2 \\ 1 \end{pmatrix} + w_2 \begin{pmatrix} 1 \\ 6 \end{pmatrix}$

Solution:

- The MATLAB code for the solution is as under:
- clear all;
  close all;
  clc;
  syms a b
  A=[-2,1;1,6];
  b=[a;b];
  Ab=[A b];
  rref(Ab)
- The possible values for the weights are $\begin{pmatrix} \frac{b}{13} - \frac{6\,a}{13} \\ \frac{a}{13} + \frac{2\,b}{13} \end{pmatrix}$

- Application of **rref** command

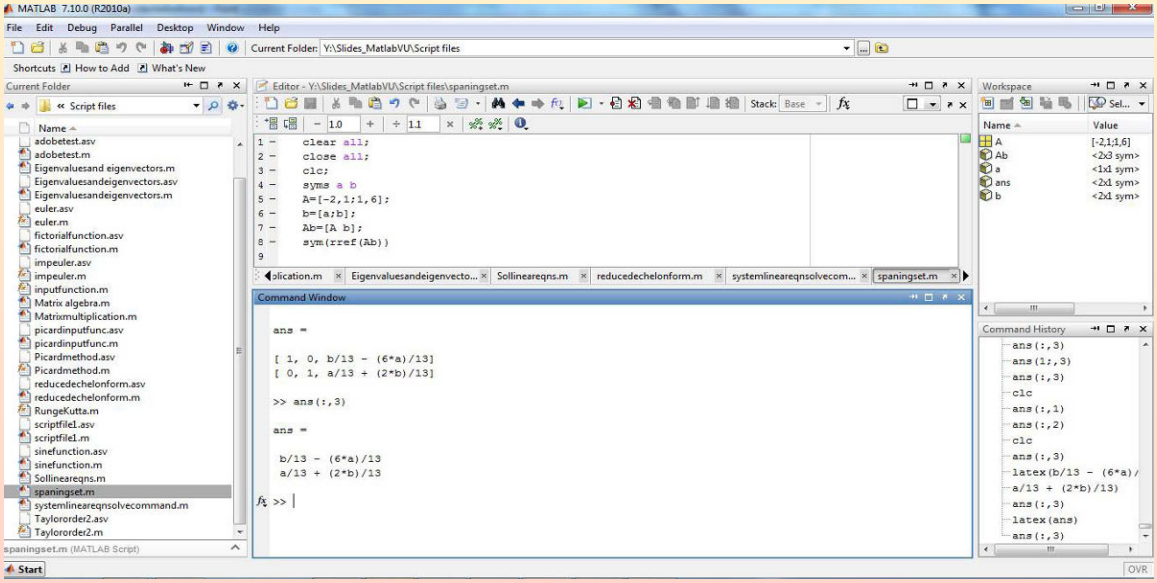


Figure: MATLAB code to determine the weights

- Consider the system of equations

$$\begin{aligned} x_1 + 2x_2 - 3x_3 &= -3, \\ -4x_1 - 5x_2 + 2x_3 &= -2, \\ 2x_1 + 3x_2 - x_3 &= 2. \end{aligned} \tag{6.1}$$

- Enter it into MATLAB as an (augmented) matrix named *Ab*.
- Use elementary row operations to reduce it to reduced echelon form.
- Give the solution of the system. (If the system has no solution say that. Otherwise, write the basic variables (those corresponding to pivot columns) as functions of numbers and free variables, if there are any.)

- Consider the system of equations

$$\begin{aligned} x_1 + 3x_3 + x_5 &= -1, \\ 4x_1 + x_2 + x_3 + 6x_5 &= 1, \\ -3x_1 - 3x_2 - 3x_3 + x_4 - 19x_5 &= 6, \\ 10x_1 - 4x_2 + 38x_3 + 2x_4 - 12x_5 &= 0. \end{aligned} \tag{6.2}$$

- Enter it into MATLAB as an (augmented) matrix named Ab.
- Use elementary row operations to reduce it to reduced echelon form.
- Give the solution of the system. (If the system has no solution say that. Otherwise, write the basic variables (those corresponding to pivot columns) as functions of numbers and free variables, if there are any.)

- Show that $\begin{pmatrix} h \\ k \end{pmatrix}$ is in the span of $\begin{pmatrix} -3 & 2 \\ 4 & 5 \end{pmatrix}$ for any h and k.

LOOPS IN MATLAB

What is a Loop?

A loop is a block of code that repeats itself

A loop can be programmed to repeat a fixed number of times or it can be programmed to repeat until some event occurs

Loop Type Comparison

- Counter Based **For Loop**
  - Runs a predetermined number of times
  - Uses a counter variable
  - Automatically updates the loop variable
- Conditional Based **While Loop**
  - Runs until some condition is met
  - Uses any variable(s)
  - Loop variable(s) must be updated manually

Both are based on a condition, it is simply that the endpoint of the counter based loop is known ahead of time (unless you make a mistake!), whereas the conditional based loop is not

General Items About Loops

- Loop condition is only checked once during each iteration
- Conditional test is based on the ***loop control variable(s)***:
  - Initialized prior entering the loop
  - Updated within the loop
  - When the loop control variable(s) take on value(s) that cause the condition to fail, the loop will exit and continue on with the rest of the program

FOR loops

FOR loops are counter based loops

Sample Construction:

```
for control_variable = begin:step:end
     MATLAB statements
end
```

- The control variable will count from the beginning value to the end value based on the step size specified
- It will repeat the statements included inside the loop (between the `for` and `end`) each time it counts
- Once the control variable is outside of the specified range, the loop will stop repeating

For Loops

- For loops are counter based loops

```
for     k=1:1:10
            MATLAB statement
end
```

Why While Loops?

- Consider the following situation:
  - Someone has been stealing your cookies! To figure out who, you are writing a script to act as a security system for your room. You need to take a measurement from a motion sensor every 10 seconds and take a picture if the motion sensor detects someone near the cookie jar.
- Can you write this program using a For loop?
  - No, you don't know if/when someone will try to steal your cookies!

WHILE loops repeat until some condition is no longer met

Construction:

```
while  conditional statement
    MATLAB statements
end
```

- As long as the condition is true, the MATLAB statements in the while loop will continue to execute
- If the expression is initially false, the while loop will never execute
- If the expression becomes false, the loop will terminate

Example:

```
while  a >= 0
     MATLAB statements
end
```

- a has to be defined in the program before you reach the while loop, otherwise the program won't run
- If a is negative, the while loop will not execute at all
- If a is positive, the while loop will continue to execute as long as it stays positive

What happens if a is positive and none of the statements in the loop ever change a?

Infinite Loop!

Example

Following code will add up each element of a vector, and display the result after ending the for loop.

```
v = 1:10;
sum = 0;
for i = 1:length(v)
   sum = sum + v(i);
end
disp(sum)
```

Example

We want to add only odd numbers from the numbers in 1:10

```
sum = 0;
for i = 1:2:10
    sum = sum + i;
end
disp(sum)
```

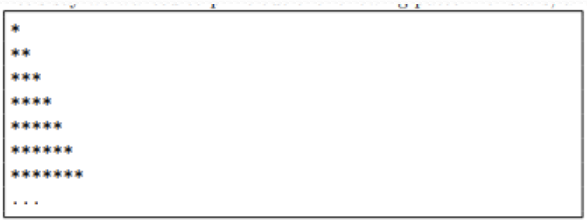
Example

Write a program to ask a number from a user, verify that the number is not negative, and compute its factorial.

```
numb=input('Enter a number: ');
fact=1;
if numb<0
fprintf('the number you have entered is negative');
else
for i=1:numb
fact=fact*i;
end
fact
end
```

Nested Loops

Let's say we wanted to print out the following pattern of stars, allowing the user to specify how many rows:



Example

```
rows=input(How many rows do you want: ');
for R=1:rows
        for s=1:R
                fprintf('*');
         end
     fprintf('\n');
end
```

Solving Differential Equations In MATLAB

Derivatives in MATLAB

```
syms x
f = sin(5*x);
```

The command

diff(f) differentiates f with respect to x:

ans = 5*cos(5*x)

To take the second derivative of g

g = exp(x)*cos(x);

diff(g,2)

Note that to take the derivative of a constant, you must first define the constant as a symbolic expression.
For example, entering

c = sym('5');
diff(c)

Derivatives of Expressions with Several Variables

To differentiate an expression that contains more than one symbolic variable, specify the variable that you want to differentiate with respect to. The diff command then calculates the partial derivative of the expression with respect to that variable. For example, given the symbolic expression

syms s t
*f = sin(s*t);*
the command
diff(f,t)
calculates the partial derivative $\partial f/\partial t$.

Integration in MATLAB

If f is a symbolic expression, then

int(f)

$$\int x^n dx = \begin{cases} \log(x) & \text{if } n = -1 \\ \dfrac{x^{n+1}}{n+1} & \text{otherwise.} \end{cases}$$

syms x

int(x^n) or int(x^n,x)

syms x
f = exp(-x^2);

int(f)
ans = (pi^(1/2)*erf(x))/2

exp(-x^2), there is no formula for the integral involving standard calculus expressions, such as trigonometric and exponential functions. In this case, MATLAB returns an answer in terms of the error function erf.

If MATLAB is unable to find an answer to the integral of a function f, it just returns int(f).

$$\int_0^{\pi/2} \sin(2x)dx = 1$$

syms x
*int(sin(2*x), 0, pi/2) or int(sin(2*x), x, 0, pi/2)*

First-Order Linear ODE

Solve this differential equation.

$dy/dt=ty$.

First, represent y by using syms to create the symbolic function y(t).

syms y(t)

Define the equation using == and represent differentiation using the diff function

*ode = diff(y,t) == t*y.*

dsolve

S = dsolve(eqn) solves the differential equation eqn, where eqn is a symbolic equation.

Use diff and == to represent differential equations. For example, diff(y,x) == y represents the equation dy/dx = y.

So we solve the equation using dsolve.

```
ySol(t) = dsolve(ode)
```

Example

Solve the first-order differential equation

dy/dt=ay

```
syms y(t) a
eqn = diff(y,t) == a*y;
S = dsolve(eqn)
```

Solve Differential Equation with Condition

In the previous solution, the constant *C* appears because no condition was specified.

Solve the equation with the initial condition *y(0) == 2*. The *dsolve* function finds a value of *C* that satisfies the condition.

```
cond = y(0) == 2;
ySol(t) = dsolve(ode,cond)
```

Example

$dy/dt+4y(t)=e^{-t}$,

$y(0)=1$.

```
syms y(t)
ode = diff(y)+4*y == exp(-t);
cond = y(0) == 1;
ySol(t) = dsolve(ode,cond)
```

Nonlinear Differential Equation with Initial Condition

Solve this nonlinear differential equation with an initial condition. The equation has multiple solutions.

$(dy/dt+y)^2=1$,
$y(0)=0$.

```
syms y(t)
ode = (diff(y,t)+y)^2 == 1;
cond = y(0) == 0;
ySol(t) = dsolve(ode,cond)
```

Second-Order ODE with Initial Conditions

- Solve this second-order differential equation with two initial conditions.

$d^2y/dx^2=cos(2x)-y$,
$y(0)=1$,
$y'(0)=0$.

```
syms y(x)
Dy = diff(y);
ode = diff(y,x,2) == cos(2*x)-y;
cond1 = y(0) == 1;
cond2 = Dy(0) == 0;
```

```
conds = [cond1 cond2];
ySol(x) = dsolve(ode,conds);
ySol = simplify(ySol)
```

Example

Solve the following differential Equation

$2x^2d^2y/dx^2+3xdy/dx-y=0.$

```
syms y(x)
ode = 2*x^2*diff(y,x,2)+3*x*diff(y,x)-y == 0;
ySol(x) = dsolve(ode)
```

Example

Solve the initial value problem $\frac{dy}{dt} = 1 + t^2 + y^2 + t^2 y^2, \quad y(0) = 1$

```
syms y(t)
eqn = diff(y,t)==1+t^2+y^2+t^2*y^2;
cond=y(0)==1;
dsolve(eqn,cond)
```

Third-Order ODE with Initial Conditions

Solve this third-order differential equation with three initial conditions.

$d^3u/dx^3 = u,$

$u(0) = 1,$

$u'(0) = -1,$

$u''(0) = \pi.$

```
syms u(x)
Du = diff(u,x);
D2u = diff(u,x,2);

ode = diff(u,x,3) == u;
cond1 = u(0) == 1;
cond2 = Du(0) == -1;
cond3 = D2u(0) == pi;
conds = [cond1 cond2 cond3];

uSol(x) = dsolve(ode,conds)
```

Solve a System of Differential Equations

Solve this system of linear first-order differential equations.

$du/dt = 3u + 4v$,

$dv/dt = -4u + 3v$.

Example

First, represent u and v by using syms to create the symbolic functions u(t) and v(t).

syms u(t) v(t)

Define the equations using == and represent differentiation using the diff function.

```
ode1 = diff(u) == 3*u + 4*v;
ode2 = diff(v) == -4*u + 3*v;
odes = [ode1; ode2]
S = dsolve(odes)
```

To access u(t) and v(t), index into the structure S.

uSol(t) = S.u

vSol(t) = S.v

Alternatively, store u(t) and v(t) directly by providing multiple output arguments.

```
[uSol(t), vSol(t)] = dsolve(odes)
```

The constants C1 and C2 appear because no conditions are specified. Solve the system with the initial conditions u(0) == 0 and v(0) == 0. The dsolve function finds values for the constants that satisfy these conditions.

```
cond1 = u(0) == 0;
cond2 = v(0) == 1;
conds = [cond1; cond2];
[uSol(t), vSol(t)] = dsolve(odes,conds)
```

Visualize the solution using fplot.

```
fplot(uSol)
hold on
fplot(vSol)
grid on
legend('uSol','vSol','Location','best')
```

Find Explicit and Implicit Solutions of Differential Equation

Solve the differential equation

$\partial/\partial t\, y(t)=e^{-y(t)}+y(t)$.

dsolve returns an explicit solution in terms of a Lambert W function that has a constant value.

```
syms y(t)
eqn = diff(y) == y+exp(-y)
sol = dsolve(eqn)
```

To return implicit solutions of the differential equation, set the 'Implicit' option to true.

```
sol = dsolve(eqn,'Implicit',true)
```

Find Implicit Solution When No Explicit Solution Is Found

If dsolve cannot find an explicit solution of a differential equation analytically, then it returns an empty symbolic array. You can solve the differential equation by using MATLAB numerical solver, such as ode45.

```
syms y(x)
eqn = diff(y) == (x-exp(-x))/(y(x)+exp(y(x)));
S = dsolve(eqn)
```

Warning: Unable to find symbolic solution.
S =[empty sym]

Alternatively, you can try finding an implicit solution of the differential equation by specifying the 'Implicit' option to true.

```
S = dsolve(eqn,'Implicit',true)
```

Numerical Solutions of ODEs in MATLAB Using ODE45

Sometimes it is not possible to find the exact solution of certain differential equations, then in this case we are forced to compute their numerical solutions.

To find the numerical solutions of the first–order ordinary differential equations, we use ode45 command.

[t,y] = ode45(odefun,tspan,yo)

where tspan = [to tf], integrates the system of differential equations $y'=f(t,y)$ from t_o to t_f with initial conditions yo. Each row in the solution array y corresponds to a value returned in column vector t.

Example

Simple ODEs that have a single solution component can be specified as an anonymous function in the call to the solver. The anonymous function must accept two inputs (t,y) even if one of the inputs is not used.

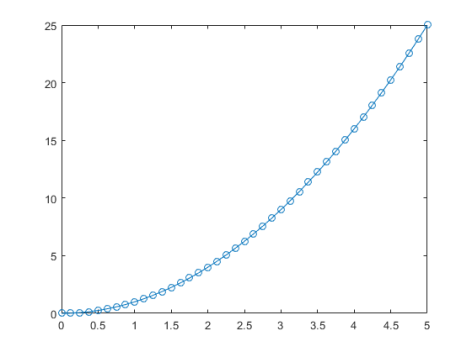
Solve the ODE

$y'=2t.$

Use a time interval of [0,5] and the initial condition yo = 0.

```
tspan = [0 5];
y0 = 0;
[t,y] = ode45(@(t,y) 2*t, tspan, y0);
```

Plot the solution.



Example

Plot the family of the approximate solutions of the following IVPs

$y'(x) = y\text{-}x^2 + 1,\ y(0) = 0{:}5 : 0{:}2 : 3$

```
f= @(x; y)y – x^2 + 1
```

```
ode45(f,[0,2],0.5:0.2:3)
```

Solve ODE with Multiple Initial Conditions

$y'(x) = -2y + 2\cos(t)\sin(2t)$

Create a vector of different initial conditions in the range [−5, 5].

Solve the equation for each initial condition over the time interval [0,3] using ode45.

```
yprime = @(t,y) -2*y + 2*cos(t).*sin(2*t);
tspan = [0 3];
y0 = -5:5;
[t,y] = ode45(yprime,tspan,y0);
```

Plot the results.

```
plot(t,y)
```

Solution of Differential Equations In MATLAB-II

Third-Order ODE with Initial Conditions

Solve this third-order differential equation with three initial conditions.

$d^3u/dx^3=u,$

$u(0)=1,$

$u'(0)=-1,$

$u''(0)=\pi.$

```
syms u(x)
Du = diff(u,x);
D2u = diff(u,x,2);

ode = diff(u,x,3) == u;
cond1 = u(0) == 1;
cond2 = Du(0) == -1;
cond3 = D2u(0) == pi;
conds = [cond1 cond2 cond3];

uSol(x) = dsolve(ode,conds)
```

Solve a System of Differential Equations

Solve this system of linear first-order differential equations.

$du/dt = 3u + 4v,$

$dv/dt = -4u + 3v.$

Example

First, represent u and v by using syms to create the symbolic functions u(t) and v(t).

syms u(t) v(t)

Define the equations using == and represent differentiation using the diff function.

```
ode1 = diff(u) == 3*u + 4*v;
ode2 = diff(v) == -4*u + 3*v;
odes = [ode1; ode2]
S = dsolve(odes)
```

To access u(t) and v(t), index into the structure S.

uSol(t) = S.u
vSol(t) = S.v

Alternatively, store u(t) and v(t) directly by providing multiple output arguments.

```
[uSol(t), vSol(t)] = dsolve(odes)
```

The constants C1 and C2 appear because no conditions are specified. Solve the system with the initial conditions u(0) == 0 and v(0) == 0. The dsolve function finds values for the constants that satisfy these conditions.

```
cond1 = u(0) == 0;
cond2 = v(0) == 1;
conds = [cond1; cond2];
[uSol(t), vSol(t)] = dsolve(odes,conds)
```

Visualize the solution using fplot.

```
fplot(uSol)
hold on
fplot(vSol)
grid on
legend('uSol','vSol','Location','best')
```

Explicit and Implicit Solutions

- An **explicit solution** is any solution that is given in the form y=y(t).

  In other words, the only place that y actually shows up is once on the left side and only raised to the first power.

- An **implicit solution** is any solution that isn't in explicit form.

Example

Let's use the example initial-value problem

$$y'y = -x, \qquad y(0) = r, \qquad r \text{ constant}$$

One can derive both an implicit and explicit solution for this DE. The *implicit* solution to this DE is

$$x^2 + y(x)^2 = r^2$$

This solution implicitly defines y(x); all we have here is an equation involving y(x). On the other hand, the explicit solution looks like

$$y(x) = \pm\sqrt{r^2 - x^2}$$

Find Explicit and Implicit Solutions of Differential Equation

Solve the differential equation

$\partial/\partial t\, y(t) = e^{-y(t)} + y(t)$.

dsolve returns an explicit solution in terms of a Lambert W function that has a constant value.

```
syms y(t)
eqn = diff(y) == y+exp(-y)
sol = dsolve(eqn)
```

To return implicit solutions of the differential equation, set the 'Implicit' option to true.

sol = dsolve(eqn,'Implicit',true)

Find Implicit Solution When No Explicit Solution Is Found

If dsolve cannot find an explicit solution of a differential equation analytically, then it returns an empty symbolic array. You can solve the differential equation by using MATLAB numerical solver, such as ode45.

```
syms y(x)
eqn = diff(y) == (x-exp(-x))/(y(x)+exp(y(x)));
S = dsolve(eqn)
```

Warning: Unable to find symbolic solution.
S =[empty sym]

Alternatively, you can try finding an implicit solution of the differential equation by specifying the 'Implicit' option to true.

S = dsolve(eqn,'Implicit',true)

Numerical Solutions of ODEs in MATLAB Using ODE45

Sometimes it is not possible to find the exact solution of certain differential equations, then in this case we are forced to compute their numerical solutions.

To find the numerical solutions of the first–order ordinary differential equations, we use ode45 command.

[t,y] = ode45(odefun,tspan,yo)

where tspan = [to tf], integrates the system of differential equations $y'=f(t,y)$ from t_o to t_f with initial conditions yo. Each row in the solution array y corresponds to a value returned in column vector t.

Example

Simple ODEs that have a single solution component can be specified as an anonymous function in the call to the solver. The anonymous function must accept two inputs (t,y) even if one of the inputs is not used.

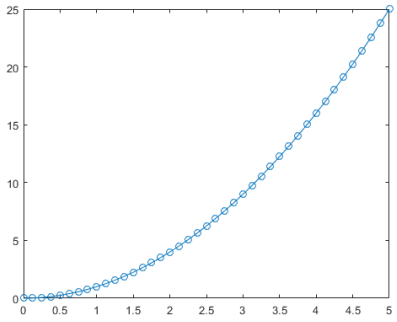
Solve the ODE

$y' = 2t$.

Use a time interval of [0,5] and the initial condition y0 = 0.

```
tspan = [0 5];
y0 = 0;
[t,y] = ode45(@(t,y) 2*t, tspan, y0);
```

Plot the solution.



Example

Plot the family of the approximate solutions of the following IVPs

$y'(x) = y - x^2 + 1,\ y(0) = 0.5 : 0.2 : 3$

```
f= @(x; y)y – x^2 + 1
```

```
ode45(f,[0,2],0.5:0.2:3)
```

Solve ODE with Multiple Initial Conditions

$y'(t) = -2y + 2\cos(t)\sin(2t)$

Create a vector of different initial conditions in the range [−5, 5].

Solve the equation for each initial condition over the time interval [0,3] using ode45.

```
yprime = @(t,y) -2*y + 2*cos(t).*sin(2*t);
tspan = [0 3];
y0 = -5:5;
[t,y] = ode45(yprime,tspan,y0);
```

Plot the results.

```
plot(t,y)
```

Numerical Solution for Differential Equation in MATLAB

Euler's Method

Euler's Method assumes our solution is written in the form of a Taylor's Series. That is, we'll have a function of the form:

$$y(x+h) \approx y(x) + hy'(x) + \frac{h^2 y''(x)}{2!} + \frac{h^3 y'''(x)}{3!} + \frac{h^4 y^{\mathrm{iv}}(x)}{4!} + \dots$$

For **Euler's Method**, we just take the first 2 terms only.

or simply $$y(x+h) \approx y(x) + hy'(x)$$

$$y(x+h) \approx y(x) + hf(x,y)$$

Continue..

The result of using this formula is the value for y, one h step to the right of the current value. Let's call it y_1. So we have:

$$y_1 \approx y_0 + hf(x_0, y_0)$$

where

y_1 is the next estimated solution value;

y_0 is the current value;

h is the interval between steps; and

$f(x_0, y_0)$ is the value of the derivative at the starting point, (x_0, y_0).

Continue…

$$y_2 \approx y_1 + hf(x_1, y_1)$$

where

y_2 is the next estimated solution value;

y_1 is the current value;

h is the interval between steps;

$x_1 = x_0 + h$; and

$f(x_1, y_1)$ is the value of the derivative at the current (x_1, y_1) point.

We continue this process for as many steps as required.

Euler Method on MATLAB

```
f=@(x,y)(x/y)
a=0;
b=2;
ya=1;
n=10;

h=(b-a)/n;      % h is the step
y=zeros(+1,1); % memory allocation, zero column vector
y(1)=ya;
    for j=1:n
    y(j+1)=y(j)+h*f(x(j),y(j)); % Euler's formula
    x(i+1)=x(i)+h;
    end
plot(x,y,'-b')
hold on
syms y(x)
ode= diff(y,x)== x/y ;
cond=y(0)==1;
S=dsolve(ode,cond)
F=matlabFunction(S)
fplot(F,[0,2],'r',"LineWidth",2);
hold off
```

Numerical Integration in MATLAB

Numerical Integration

Sometimes, the evaluation of expressions involving the integrals can become daunting, if not indeterminate. For this reason, a wide variety of numerical methods has been developed to simplify the integral.

Trapezoidal Rule

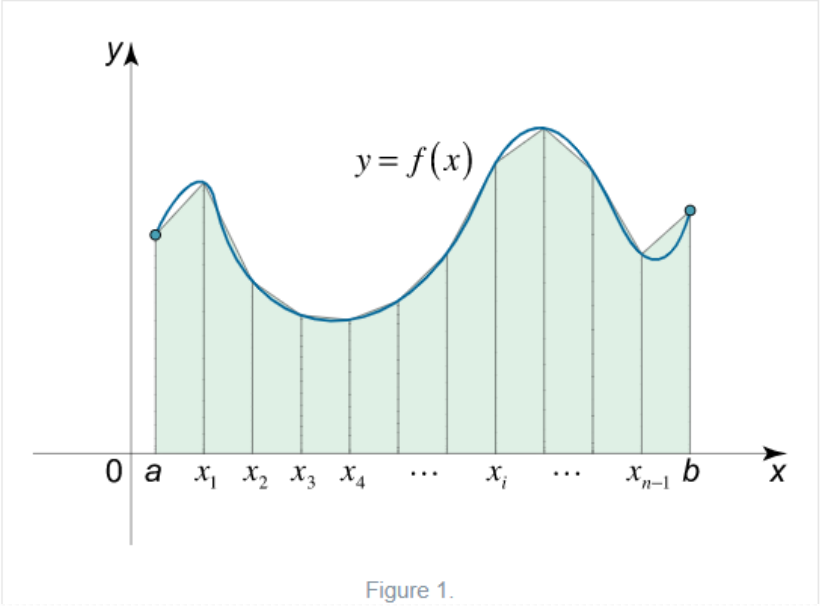


Figure 1.

The Trapezoidal Rule for approximating $\int_a^b f(x)\,dx$ is given by

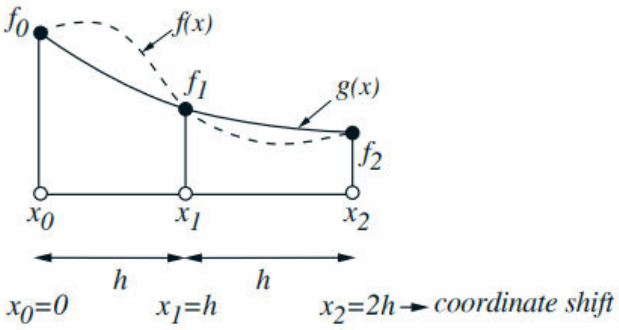
$$\int_a^b f(x)\,dx \approx T_n = \frac{\Delta x}{2}\left[f(x_0) + 2f(x_1) + 2f(x_2) + \cdots + 2f(x_{n-1}) + f(x_n)\right],$$

where $\Delta x = \frac{b-a}{n}$ and $x_i = a + i\Delta x$.

Code

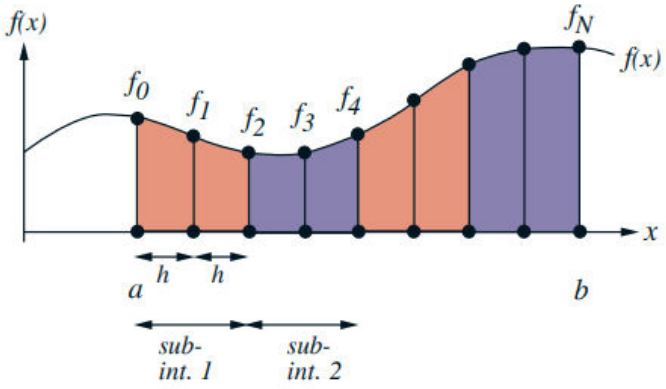
```
clc;
clear all;
f=@(x)1/(1+x); %Change here for different function
 a=input('Enter lower limit a: '); % exmple a=1
 b=input('Enter upper limit b: ');  % exmple b=2
 n=input('Enter the no. of subinterval: ');  % exmple n=10
 h=(b-a)/n;
 sum=0;
for k=1:1:n-1
  x(k)=a+k*h;
  y(k)=f(x(k));
  sum=sum+y(k);
end
% Formula:  (h/2)*[(y0+yn)+2*(y2+y3+..+yn-1)]
answer=h/2*(f(a)+f(b)+2*sum);
fprintf('\n The value of integration is %f',answer);  % exmple The value of integration is
0.410451
```

Simpson's 1/3 Rule



$$I = \frac{h}{3}[f_0 + 4f_1 + f_2]$$

Continue…



Continue…

$$I = \int_a^b f(x)dx$$

$$\Rightarrow$$

$$I = \frac{h}{3}((f_o + 4f_1 + f_2) + (f_2 + 4f_3 + f_4) + (f_4 + 4f_5 + f_6)$$

$$+ \ldots + (f_{N-4} + 4f_{N-3} + f_{N-2}) + (f_{N-2} + 4f_{N-1} + f_N)) + E_{[a,b]}$$

$$\Rightarrow$$

$$I = \frac{h}{3}[f_o + 4f_1 + 2f_2 + 4f_3 + 2f_4 + 4f_5 + \ldots + 2f_{N-2} + 4f_{N-1} + f_N]$$

Continue

$$\int_a^b f(x)\,dx \approx \frac{h}{3}\sum_{j=1}^{n/2}\left[f(x_{2j-2}) + 4f(x_{2j-1}) + f(x_{2j})\right]$$

$$= \frac{h}{3}\left[f(x_0) + 2\sum_{j=1}^{n/2-1} f(x_{2j}) + 4\sum_{j=1}^{n/2} f(x_{2j-1}) + f(x_n)\right],$$

```
clc;
clear all;
f=@(x)x^3; %Change here for different function
a=input('Enter lower limit a: '); % exmple a=0
b=input('Enter upper limit b: '); % exmple b=3
n=input('Enter the number of sub-intervals n: '); % exmple n=16
h=(b-a)/n;
if rem(n,2)==1
fprintf('\n Enter valid n!!!');
n=input('\n Enter n as even number ');
End
so=0;se=0;

for k=1:1:n-1
x(k)=a+k*h;
for k=1:1:n-1
if rem(k,2)==1
so=so+f(x(k));%sum of odd terms
else
se=se+f(x(k)); %sum of even terms
end
end
answer=h/3*(f(a)+f(b)+4*so+2*se);
fprintf('\n The value of integration is %f',answer);
```

Numerical Integration in MATLAB

Simpson's 3/8 Rule

$$\int_a^b f(x)\,dx \approx \frac{3h}{8}\left[f(a) + 3f\left(\frac{2a+b}{3}\right) + 3f\left(\frac{a+2b}{3}\right) + f(b)\right]$$

$$= \frac{3h}{8}\left[f(x_0) + 3\sum_{i\neq 3k}^{n-1} f(x_i) + 2\sum_{j=1}^{n/3-1} f(x_{3j}) + f(x_n)\right] \quad \text{for } k \in \mathbb{N}_0.$$

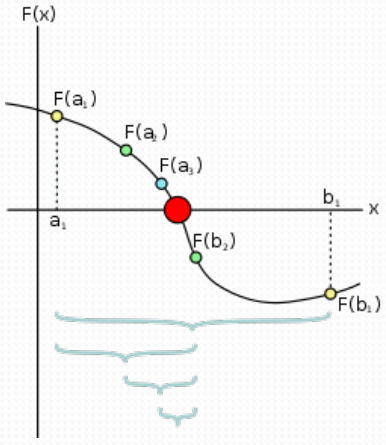
Continue…

```
clc;
clear all;
f=@(x)1/(1+x); %Change here for different function
 a=input('Enter lower limit a: '); % exmple a=1
 b=input('Enter upper limit b: ');  % exmple b=2
 n=input('Enter the  number of sub-intervals n: '); % exmple n=21
 h=(b-a)/n;
if rem(n,3)~=0
   fprintf('\n Enter valid n!!!');
   n=input('\n Enter n as multiple of 3: ');
end
for k=1:1:n
  x(k)=a+k*h;
  y(k)=f(x(k));
end
 s0=0;sm3=0;
 for k=2:1:n-1
    if rem(k,3)==0
       sm3=sm3+y(k); %sum of multiple of 3 terms
     else
     s0=s0+y(k);%sum of others terms
    end
end
answer=(3*h/8)*(f(a)+f(b)+3*s0+2*sm3);
fprintf('\n The value of integration is %f',answer); % exmple The value of integration is 0.381665
```

Bisection Method in MATLAB

Bisection Method

The method is applicable for numerically solving the equation f(x) = 0 for the real variable x, where f is a continuous function defined on an interval [a, b] and where f(a) and f(b) have opposite signs.



Continue…

At each step the method divides the interval in two by computing the midpoint c = (a+b) / 2 of the interval and the value of the function f(c) at that point.

Unless c is itself a root, there are now only two possibilities: either f(a) and f(c) have opposite signs and bracket a root, or f(c) and f(b) have opposite signs and bracket a root. The method selects the subinterval that is guaranteed to be a bracket as the new interval to be used in the next step.

Iterative Scheme

The input for the method is a continuous function f, an interval [a, b], and the function values f(a) and f(b). The function values are of opposite sign. Each iteration performs these steps:

- Calculate c, the midpoint of the interval,

c = a + b/2.

- Calculate the function value at the midpoint, f(c).
- If convergence is satisfactory (that is, c - a is sufficiently small, or |f(c)| is sufficiently small), return c and stop iterating.
- Examine the sign of f(c) and replace either (a, f(a)) or (b, f(b)) with (c, f(c)) so that there is a zero crossing within the new interval.

Example

Suppose that the bisection method is used to find the root of the polynomial.

$$f(x) = x^3 - x - 2$$

First, two numbers a and b have to be found such that f(a) and f(b) have opposite signs. For the above function, a=1 and b=2 satisfy this criterion, as

f(1)=-2
f(2)=4

Because the function is continuous, there must be a root within the interval [1, 2].

In the first iteration, the end points of the interval which brackets the root are a1=1 and b1=2, so the midpoint is

c= (a+b)/2 = 1.5

f(c)=-0.125

Because f(c) is negative, a=1 is replaced with a=1.5

Code

```
f=@(x)x^3-x-2;
a=1;
b=2;
 if f(a)*f(b) > 0
    disp('There is no change of sign');
    return
end
 tol=10^-5;
disp('Iter.   a          b        c');
i=1;
while(abs(a-b)>=tol)
c=(a+b)/2;
if(f(c)==0)
       fprintf('Root of function is %f \n',c)
            return
    end
fprintf('%2i \t %f \t %f \t %f \n',i, a, b, c)
    if(f(a)*f(c)>0)
        a=c;
    else
        b=c;
    end
i=i+1;
end
m=(a+b)/2
fprintf('Root lies at c= %f \n',m )
```